Week 03 – Prompting frameworks for text-to-SQL

Model: `gpt-5.6-luna`, low reasoning, run as a coding agent (one fresh CLI session per prompt).

Task 1 – Frameworks and prompts

Prompts: `agents/<name>/prompt.txt`.

Framework	Parts	Why chosen	Expectation
RTF	Role, Task, Format	Shortest structure - the task is simple.	Should cause the agent less confusion
ICIO	Instruction, Context, Input, Output	Separates inputs from output. Fits this as a data processing task	Fewer mistakes about what to read and write.
CO-STAR	Context, Objective, Style, Tone, Audience, Response	Tells the model the HOW and for WHO we are doing this	More structured output.

All the prompts include a reference to two hand written examples in `inputs/examples.md`. I found all of these frameworks on https://frompting.com/frameworks.

I decided to lean into the agentic capabilities of the harness I used (codex) and encouraged the agents to test their queries and fix the ones that are wrong.

I also told the agent to stop and report when progress becomes impossible, which later turned out to be useful due to question no.65

Task 2 – Results

Prompt	Predictions	Prompt file	Model	Reasoning
RTF	`agents/rtf/predictions.csv`	`agents/rtf/prompt.txt`	`gpt-5.6-luna`	low
ICIO	`agents/icio/predictions.csv`	`agents/icio/prompt.txt`	`gpt-5.6-luna`	low
CO-STAR	`agents/co_star/predictions.csv`	`agents/co_star/prompt.txt`	`gpt-5.6-luna`	low

Full session record (model, timing, tokens, every agent event): `agents/<name>/run/` (`metadata.json`, `events.jsonl`). All checked attempts: `agents/<name>/attempts/`.

Prompt	First attempt	Final	Checker calls	Session time
RTF	32/155 (21%)	141/155 (91%)	9	117 s
ICIO	93/155 (60%)	141/155 (91%)	10	212 s
CO-STAR	76/155 (49%)	**147/155 (95%)**	9	121 s

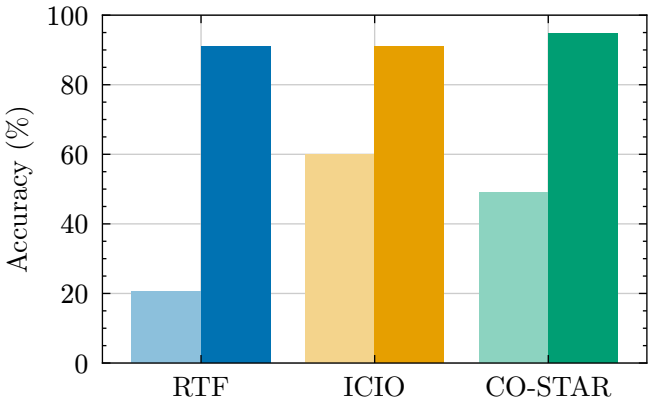


Figure 1: Accuracy: first attempt (light) vs final (solid).

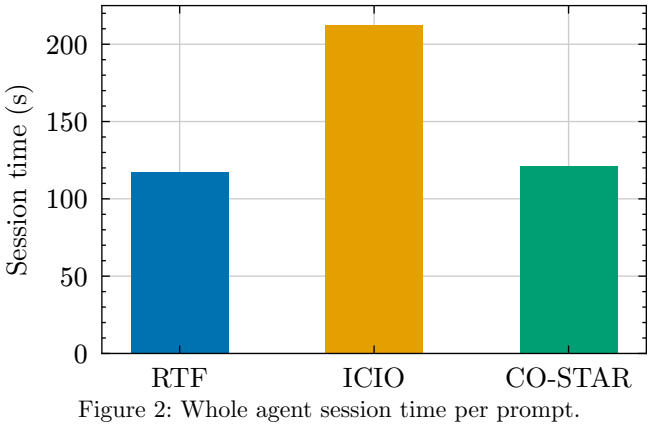


Figure 2: Whole agent session time per prompt.

Token usage per session (cached ⊂ input, reasoning ⊂ output):

Prompt	Input tokens	cached	Output tokens	reasoning
RTF	682,539	635,136	10,167	2,470
ICIO	1,364,353	1,308,672	13,613	4,337
CO-STAR	1,332,809	1,272,576	10,501	3,205

- CO-STAR finished best. ICIO started best. RTF was fastest.
- Time covers the whole agent session: reading, coding, checking and repairing.
- None reached 155/155. In part this is because of the trick question no.65 which wasn't a real question. All three noticed it; ICIO named it in its final report.
- Agents wrote a Python script that generates the SQL, then repaired it using checker feedback - this was not prompted in, and it seems agents did this on their own. It seems they prefer using custom scripts for doing tasks like this. The script tries to parse the human written questions and automatically generate sql for it. If this was a real application and we actually got similar csv files on the input, it might be quite a good idea to have some fast deterministic prepass which might knock out most of the questions, and over time it could be improved so that the model only needs to handle the edge-cases by itself.

Task 3 – Added metric: balanced result F1

Exact accuracy marks an answer wrong if even one row is missing or extra. F1 gives partial credit.

Each query returns a list of rows (restaurants). We compare it with the rows of the correct answer and count:

- **TP** (true positives): correct rows the query returned
- **FP** (false positives): wrong rows the query returned
- **FN** (false negatives): correct rows the query missed

$$\mathrm{F1} = \frac{2 \cdot \mathrm{TP}}{2 \cdot \mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

F1 is 1 for a perfect answer and 0 when no returned row is correct. It is a standard metric in information retrieval and machine learning (C. J. van Rijsbergen, *Information Retrieval*, 1979; scikit-learn `f1_score`).

Balanced: 43 of the 155 questions have an empty correct answer, so a query that returns nothing gets those right for free. We average F1 over those 43 questions and over the other 112 separately, then take the mean of the two.

This metric complements the original exact accuracy. We might prefer a model that has a few big mistakes but most things correct than a model that has a small mistake almost everywhere, so using F1 and the exact metric gives us more information than using either individually.

Code: `analysis/src/metrics.py`. Run: `python3 analysis/src/analyze.py`. Per-question scores: `analysis/results/<name>.per_question.csv`.

Prompt	First	Final
RTF	0.375	0.969
ICIO	0.653	0.939
CO-STAR	0.629	**0.992**

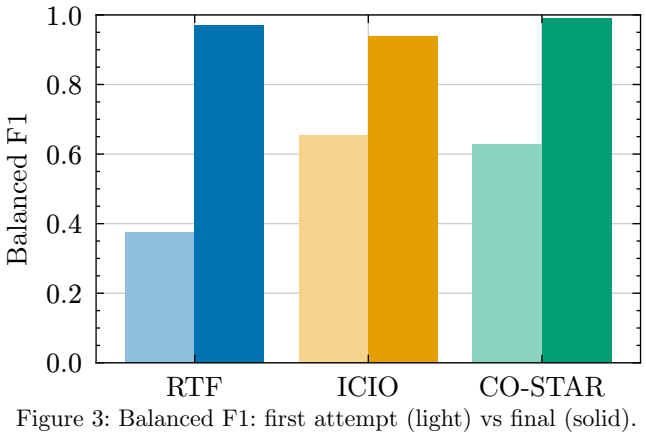


Figure 3: Balanced F1: first attempt (light) vs final (solid).

We can see that the first metric tied RTF and ICIO on accuracy, but looking at F1 we can see that RTF's wrong answers are closer to correct at the end, though not significantly.

Task 4 – Analysis

Progress over checker attempts

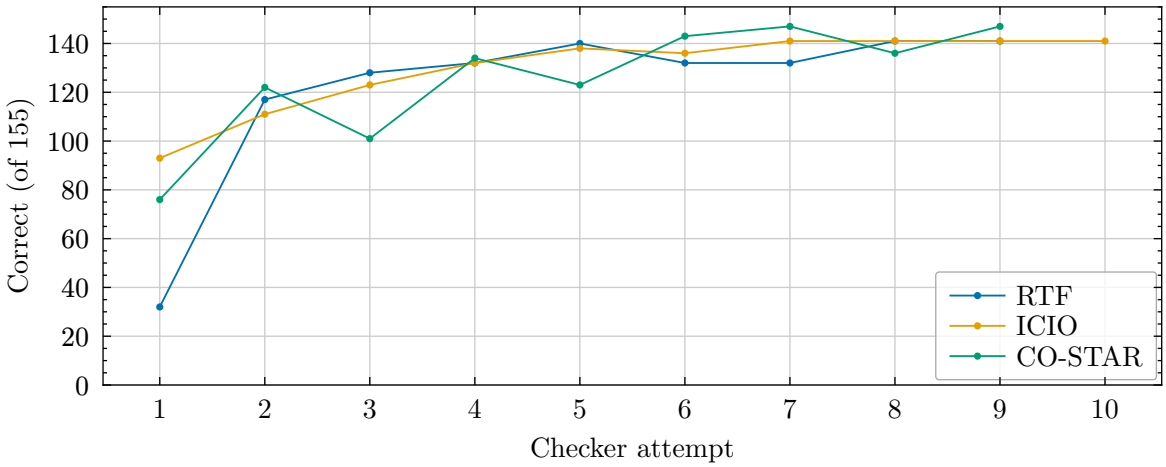


Figure 4: Correct answers after each checker call. Drops show repairs that made things worse.

All prompts improved most in the first few attempts. Some repairs broke questions that had passed before, so scores sometimes dropped. Interestingly the most successful prompt (CO-STAR) swings in accuracy the most. It may indicate that the model took more risk which turned out successful in this case.

Which questions failed

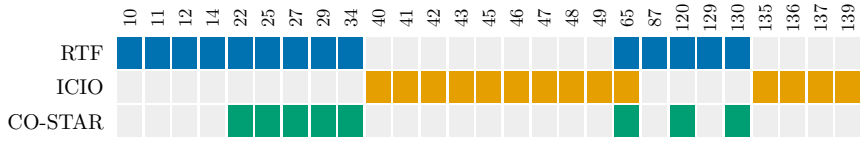


Figure 5: Failed questions in the final predictions (filled = failed). Columns are question IDs failed by at least one prompt.

- 27 distinct questions failed in at least one final result.
- Failed by all three: 65.
- ICIO's failures sit in other ID blocks than RTF's and CO-STAR's, so its script had different blind spots. It would be an interesting experiment to try and run ICIO and CO-STAR in parallel, and then have one agent merge their solutions taking the successful ones from both.

Is the difference significant?

Tests: McNemar (exact) for accuracy, paired permutation test for balanced F1, Holm correction for 3 comparisons, $\alpha = 0.05$. Data: `analysis/results/statistics.json`.

	Accuracy (McNemar)			Balanced F1 (permutation)		
A vs B	**Only A / only B**	**p**	**Sig.**	**A − B**	**p**	**Sig.**
Final results						
RTF vs ICIO	13 / 13	1	no	0.029	0.17	no
RTF vs CO-STAR	0 / 6	0.09	no	−0.023	0.006	**yes**
ICIO vs CO-STAR	7 / 13	0.53	no	−0.052	0.006	**yes**
First attempts						
RTF vs ICIO	12 / 73	< 0.001	**yes**	−0.278	< 0.001	**yes**
RTF vs CO-STAR	4 / 48	< 0.001	**yes**	−0.254	< 0.001	**yes**
ICIO vs CO-STAR	31 / 14	0.02	**yes**	0.024	0.63	no

Only A / only B: questions only A / only B got right. *A − B:* difference in balanced F1. *p:* p-value after Holm correction. *Sig.:* $p < 0.05$.

- Final, accuracy: no significant difference.
- Final, balanced F1: CO-STAR significantly better than both.
- First attempts: all accuracy differences significant (ICIO > CO-STAR > RTF).

Why the differences

Prompt

- CO-STAR is the longest and most explicit.
- RTF is shortest. Its first script was the weakest (32/155).

Model (same for all, low reasoning)

- Wrote one regex-based parser that translates all questions with the same rules.
- Fitted fixes to checker feedback; some repairs made scores worse (CO-STAR 147 → 136 → 147).

Coding agent

- Results depend mostly on each script's parsing choices:
  - RTF: hardcoded list of 5 restaurant names.
  - ICIO: read the database to build city and name lists, which was good, but it also patched one question (ID 28) by ID.
  - CO-STAR: its parsing handled the most questions, so it scored the most points.
- Every script created by the model is really unreadable. I wouldn't want to be the one left to maintain this.
- ID 65 ("What is the lifes greates illusion?") was a control against hardcoded answers. All three failed it, which suggests none hardcoded the expected result rows.

Limits

- One run per framework, so run-to-run variation is unknown.
- Different scripts and repairs mean differences can't be attributed to prompt wording alone. I wouldn't consider any of these results conclusive at all.

AI assistance

The analysis scripts and visualizations were co-written with Claude Opus 5.5 and GPT-6.1 Sol.